

Internalizing Algebraic Symmetry in Carry Dynamics via Encode-Process-Decode Algorithmic Networks

Shakunth¹

Department of Artificial
Intelligence and Machine learning
St. Joseph’s College of Engineering
Chennai, India
shakunth.s@stjosephs.ac.in

Thomas Albert Iwin²

Department of Artificial
Intelligence and Machine learning
St. Joseph’s College of Engineering
Chennai, India
thomas.albert.iwin@stjosephs.ac.in

Poornima Ravi³

Department of Artificial
Intelligence and Machine learning
St. Joseph’s College of Engineering
Chennai, India
poornima@stjosephs.ac.in

Abstract—Neural networks struggle with tasks requiring strict adherence to discrete logical invariants, particularly on out-of-distribution (OOD) sequence lengths. We investigate carry propagation in symmetric digit-reversal multiplication ($n \cdot \text{rev}(n)$), governed by a global algebraic symmetry that must be preserved for palindromic output. We characterize an Accuracy-Reasoning Gap: standard sequential (RNN) and attention-based (Transformer) models achieve misleadingly high token-level accuracy via local heuristics but violate the underlying algebraic constraints. We propose the Encode-Process-Decode Graph Neural Network (EPD-GNN), which employs iterative latent execution with explicit symmetry-based topological priors. Evaluations on sequences up to $5\times$ longer than training samples show that EPD-GNN achieves superior task accuracy with significantly lower constraint violation compared to sequential and parallel baselines. Our results suggest that structural priors are essential for robust neural algorithmic reasoning beyond the interpolation regime.

Index Terms—graph neural networks, neural computing, arithmetic, inductive reasoning, recurrent neural networks.

I. Introduction

NEURAL Algorithmic Reasoning (NAR) has emerged as a transformative subfield within deep learning, striving to bridge the persistent gap between statistical pattern matching and discrete logical execution [1]. The fundamental objective of NAR is to design and train neural architectures that can internalize the step-by-step causal logic of classical algorithms.

Contributions. (1) We formalize the Algebraic Symmetry Constraint for carry propagation in palindromic multiplication and introduce the Conditional Algebraic Violation (CAV) as a robust evaluation metric. (2) We propose EPD-GNN with symmetry-aware topological priors and demonstrate its advantage over sequential and attention-based baselines under OOD length extrapolation. (3) We provide causal evidence via edge masking and rewiring experiments that symmetry edges—not graph density—drive generalization.

Unlike standard supervised learning, which typically relies on approximating continuous statistical manifolds within high-dimensional spaces, algorithmic tasks require

an architecture to operate with absolute precision over discrete state spaces. A minor perturbation or a single incorrect intermediate state in an algorithmic trajectory can lead to significant failure, violating the strict invariants that govern the procedure.

A fundamental challenge in the pursuit of generalized neural reasoning is the “Accuracy-Reasoning Gap”. This phenomenon occurs when a neural network achieves misleadingly high token-level accuracy on the training distribution by identifying and exploiting superficial local heuristics, correlational artifacts, or bounded memorization strategies. When such models are subsequently evaluated on out-of-distribution (OOD) samples—most notably sequences that are significantly longer or structurally more complex than those encountered during training—their performance typically degrades sharply. This degradation suggests that the model never learned the underlying generative algorithm; rather, it learned a bounded approximation that only holds within the interpolation regime.

A fundamental requirement for robust, generalized Artificial Intelligence (AI) systems is the capability to learn and execute operations exhibiting strict algebraic symmetry. The ability to abstract these operations to generic sequence lengths and arbitrary scales dictates true programmatic understanding. In this paper, we undertake a comprehensive study of the deeply recursive and mathematically constrained problem of carry propagation, specifically situated within the domain of symmetric, or palindromic, multiplication ($P = n \cdot \text{rev}(n)$). This task is uniquely and ideally suited for benchmarking NAR because it is governed by a strict, quantifiable algebraic invariant that must hold universally for the output sequence to maintain palindromic symmetry.

Carry propagation is arguably the most fundamental yet non-trivial example of a recursive, state-dependent operation in computer science and arithmetic. The standard addition of two large integers, and by extension their multiplication, forces a strict information bottleneck: the calculation at the most significant digit position

depends causally and absolutely on the sequence of carries propagating sequentially from the least significant digit. This creates a deeply recurrent dependency chain spanning the entire length of the operands. If a neural model can internalize carry dynamics across arbitrary lengths, it demonstrates a verifiable capability to reason algorithmically over extended sequences, effectively simulating long-range algorithmic dependencies within its latent space.

Historically, standard Sequence-to-Sequence models, such as Long Short-Term Memory networks (LSTMs) and standard Transformers, suffer heavily from OOD length generalization failures [12]. To combat this, the research community has increasingly turned towards Graph Neural Networks (GNNs) [2], which can be equipped with powerful algorithmic inductive biases. These models are typically framed within an Encode-Process-Decode (EPD) loop, allowing for iterative latent execution steps that naturally mirror the loops of classical algorithms. However, ensuring that these models capture deep algebraic invariants—rather than exploiting shallow sequence symmetries—remains a formidable and open challenge.

Our specific focus on palindromic multiplication is deliberate. When an integer n is multiplied by its digit-reversed counterpart $rev(n)$, the underlying continuous convolution of their digits is inherently symmetric. However, the discrete base- b resolution of this convolution introduces non-linear carries that temporarily break this symmetry locally, only to restore it globally in the final product. This restoration implies a rigid mathematical constraint on the carry sequence itself. By evaluating how well a neural network adheres to this constraint (the “Algebraic Constraint Violation”), we can measure the extent to which the model has internalized the algorithm, as opposed to merely measuring its token-level accuracy.

II. Related Work

The challenge of teaching neural networks to reason algorithmically sits at the intersection of several critical subfields of artificial intelligence, including graph theory, arithmetic learning, and formal verification. In this section, we review the foundational literature in NAR, length extrapolation, and the structural requirements for algorithmic alignment.

A. Neural Algorithmic Reasoning Foundations

The term “Neural Algorithmic Reasoning” was formalized by Veličković et al. [1], who argued that architectures should be designed to mirror the data structures and control flows of classical algorithms. They demonstrated that Graph Neural Networks (GNNs) are particularly well-suited for this task because their message-passing operations can be interpreted as discrete execution steps on a computational graph. Their work emphasized the concept of “Algorithmic Alignment,” which states that a model generalizes better when its internal computation graph matches the structure of the target algorithm. Unlike

their work, which focuses on classical graph algorithms, we apply this principle to internalize recursive algebraic invariants in arithmetic.

Early attempts to bridge the gap between connectionism and symbolic logic included the Neural Turing Machine (NTM) proposed by Graves et al. [4]. The NTM utilized a standard recurrent network augmented with an external memory bank, which it could access through a differentiable attention mechanism. While groundbreaking, the NTM was notoriously unstable to train. Our approach avoids the instability of external memory by encoding algorithmic priors directly into the graph topology.

Vinyals et al. [5] introduced Pointer Networks, which addressed a specific class of algorithmic problems where the output is a permutation of the input. While Pointer Networks are restricted to indexing existing tokens, our framework generates new, state-dependent carry tokens through non-linear latent execution.

B. Length Extrapolation and the Transformer Bottleneck

Length extrapolation—the ability of a model to generalize from training sequences of length N to testing sequences of length $M \gg N$ —is a major hurdle for contemporary models. Standard Transformers utilize positional encodings that define a fixed maximum context window, leading to collapse on OOD lengths [12]. We move beyond relative encodings by employing a recurrent EPD-GNN that generalizes via stationary execution steps rather than positional shifting.

C. Graph Neural Networks and Structural Alignment

Xu et al. [2] provided a theoretical analysis of GNN expressive power, proving that standard message-passing GNNs are at most as powerful as the Weisfeiler-Lehman test. We augment GNN expressive power by injecting task-specific symmetry arcs that reduce the topological diameter for global invariant verification.

Li et al. [6] introduced Gated Graph Neural Networks (GGNNs), which replace simple message aggregation with GRU updates. While GGNNs provide a general framework for graph-based sequences, our work specifically aligns the message-passing directions and symmetry arcs with the algebraic structure of carry dynamics. Santoro et al. [7] proposed a relational reasoning module that improves pairwise object reasoning, though it does not address the sequential positional dependencies in carry propagation.

We additionally compare against Graph Attention Networks (GAT) [13] and the ALiBi-Transformer [12]. Our benchmark differs by isolating the non-linear execution step of carry propagation and evaluating it against a necessary algebraic condition on carry propagation (the Algebraic Symmetry Constraint). Unlike standard multiplication datasets, we decouple the simpler task of convolution from the recursive logic of carries. This allows us to identify the “Accuracy-Reasoning Gap,” where models

achieve high accuracy on short sequences while fundamentally violating the underlying mathematical laws.

Gilmer et al. [8] formalized the Message Passing Neural Network (MPNN) framework. Battaglia et al. [9] expanded this into the “Relational Inductive Biases” framework, arguing that the success of deep learning depends on matching the inductive bias of the architecture to the symmetry properties of the problem. Our work extends this principle by deriving a necessary algebraic condition on carry propagation and mapping it onto the relational inductive bias of a GNN.

Arithmetic has long been a litmus test for neural reasoning capacity. Trask et al. [11] introduced Neural Arithmetic Logic Units (NALUs) to solve scalar arithmetic generalization. NALUs use a specialized gate mechanism to learn linear and logarithmic operations, allowing them to perform addition and multiplication on scalar values. While NALUs improved performance on simple 1-digit or 2-digit operations, they lacked the recursive depth needed for long-digit carry propagation.

Kaiser et al. [10] introduced the Neural GPU, which uses 2D convolutional filters to simulate a multi-step computation canvas. The Neural GPU was designed to handle sequences of arbitrary length by applying the same filters repeatedly. While it showed impressive results on binary addition, it was highly sensitive to hyperparameters and lacked a mechanism to enforce global algebraic constraints. The model often failed to maintain the mathematical integrity of the result as the sequence length increased.

The concept of carry-weight identity establishes that the sum of the digits of a product is directly linked to the total carry weight. Specifically, the sum of the digits of the product of n and its reverse is equal to the square of the sum of the digits of n , minus a factor proportional to the total carry. Our research builds on this by identifying the specific regime of “carry-driven palindrome recovery.” In this regime, the non-linear carries break local symmetry but restore global palindromicity through a delicate balancing act across the entire sequence. Benchmarking models against this recovery requires a rigorous understanding of the underlying algebraic constraints, which we formalize in Section III.

III. Mathematical Formalization

We characterize the structural properties of digit-reversal multiplication. Let n be an integer represented in base b by the sequence of digits $D = (d_0, d_1, \dots, d_k)$, where d_0 is the least significant digit. The digit-reversed sequence is $rev(D) = (d_k, d_{k-1}, \dots, d_0)$. The product $P = n \cdot rev(n)$ has a final sequence length $L \approx 2k + 1$.

A. The Convolutional Core and Non-Linear Resolution

The multiplication operation can be decomposed into two distinct phases: a linear convolution phase and a non-linear carry resolution phase. The coefficients A_i of the product, prior to any carry propagation, represent the

discrete linear convolution of the digit sequence with its reversal:

$$A_i = \sum_{j=0}^k d_j \cdot d_{k-i+j} \quad (1)$$

Because the operation involves a sequence convoluted with its exact reverse, these coefficients naturally exhibit perfect symmetry around the center of the sequence:

$$A_i = A_{L-1-i} \quad \forall i \in \{0, 1, \dots, L-1\} \quad (2)$$

This underlying linear symmetry forms the geometric foundation of the problem. However, in any base- b number system, the final digits of the product must be bounded between 0 and $b-1$. The actual digits R_i of the product are derived by incorporating the non-linear carry terms C_i that propagate from the least significant digit to the most significant digit:

$$R_i = (A_i + C_i) \pmod{b} \quad (3)$$

$$C_{i+1} = \left\lfloor \frac{A_i + C_i}{b} \right\rfloor \quad (4)$$

where $C_0 = 0$. The introduction of these non-linear carries C_i temporarily breaks the inherent symmetry of the A_i sequence. For the final product to be a palindrome, the carry sequence must restore this symmetry globally.

B. Carry Regimes and Mathematical Invariants

We analyze the behavior of palindromic multiplication through the lens of the digit sum squared, $E(n) = \sum_{j=0}^k d_j^2$.

Theorem 1 (Linear-Convolutional Regime). If $E(n) < b$, the product $P = n \cdot rev(n)$ is a perfect palindrome, and the carry sequence is zero ($C_i = 0$ for all i).

Proof. The convolution coefficients are $A_i = \sum_{j=0}^k d_j d_{k-i+j}$. By the Cauchy-Schwarz inequality, the absolute value of the inner product of two vectors is bounded by the product of their norms: $|\langle \mathbf{d}, \mathbf{d}_{\text{shifted}} \rangle| \leq \|\mathbf{d}\|_2 \|\mathbf{d}_{\text{shifted}}\|_2$. Since the shift operation rearranges or zero-pads the entries of $\mathbf{d}$ without introducing new nonzero values, $\|\mathbf{d}_{\text{shifted}}\|_2 \leq \|\mathbf{d}\|_2$. It follows that $A_i \leq \sum_{j=0}^k d_j^2 = E(n)$ for all i .

If $E(n) < b$, then $A_i \leq b-1$ for all i . We prove $C_i = 0$ by induction. Base case: $C_0 = 0$ by definition. Inductive step: assume $C_i = 0$. Then $C_{i+1} = \lfloor (A_i + C_i)/b \rfloor = \lfloor A_i/b \rfloor$. Since $0 \leq A_i \leq b-1$, it follows that $C_{i+1} = 0$. Thus, $C_i = 0$ for all i . The output digits are then $R_i = (A_i + C_i) \pmod{b} = A_i$. Since A_i is symmetric, P is a palindrome. $\square$

Proposition 2 (Algebraic Symmetry Constraint). For a product P to be a palindrome ($R_i = R_{L-1-i}$), the carries $\{C_i\}$ and coefficients $\{A_i\}$ must satisfy:

$$b(C_{i+1} - C_{L-i}) = (A_i - A_{L-1-i}) + (C_i - C_{L-1-i}) \quad (5)$$

for all $0 \leq i < L/2$.

Proof. Substituting the resolution identity $R_j = A_j + C_j - bC_{j+1}$ into the symmetry condition $R_i = R_{L-1-i}$ and rearranging terms to isolate the base b yields the constraint. $\square$

Equation (5) defines a necessary condition for palindromicity. While sufficiency is generally assumed in the integer domain where carries are derived from A_i , the constraint ensures that the global relationship between carries at opposite ends of the sequence is maintained.

We define the Algebraic Constraint Violation (V_i) as the absolute difference between the left-hand and right-hand sides of (5). If a network understands the symmetry of carries, it will produce sequences where $V_i \approx 0$. This metric is the primary evaluator of whether a model has truly internalized the algebraic structure of the algorithm.

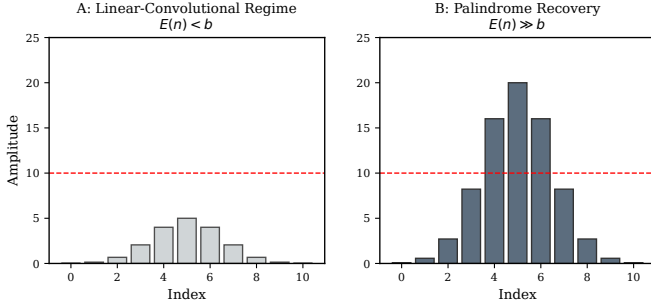


Fig. 1: The two distinct regimes of palindromic multiplication. Panel A shows the trivial Linear-Convolutional Isomorphism, where no carries are generated. Panel B shows carry-driven palindrome recovery, where massive non-linear cascades mathematically cancel out to restore global symmetry.

Algorithm 1 EPD-GNN Forward Pass

Require: Input sequence $X = \{(d_i, A_i)\}$, graph topology $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, processing steps T_{exec}
 Ensure: Predicted carries $\{\hat{C}_i\}$, palindrome probability $\hat{y}$

```

1:  $H^{(0)} \leftarrow \text{MLP}_{enc}(X)$  {Initial node embeddings}
2: for  $t = 1$  to  $T_{exec}$  do
3:   for node  $i \in \mathcal{V}$  do
4:      $M_i^{(t)} \leftarrow \sum_{j \in \mathcal{N}(i)} \text{MLP}_{msg}(h_i^{(t-1)} \parallel h_j^{(t-1)} \parallel e_{ij})$ 
5:      $h_i^{(t)} \leftarrow \text{GRU}(h_i^{(t-1)}, M_i^{(t)})$ 
6:   end for
7: end for
8:  $\{\hat{C}_i\} \leftarrow \text{MLP}_{carry}(H^{(T_{exec})})$ 
9:  $\hat{y} \leftarrow \sigma(\text{MLP}_{pal}(\text{GlobalMean}(H^{(T_{exec})})))$ 
10: return  $\{\hat{C}_i\}, \hat{y}$ 

```

The proposed system utilizes an Encode-Process-Decode (EPD) Graph Neural Network architecture designed to align its internal logic with the algebraic invariants described in Section III. The EPD paradigm is

chosen because it separates the task of representing the data (Encoding) from the task of executing the algorithm (Processing) and generating the output (Decoding).

C. Data Preprocessing and Feature Extraction

The input at each sequence index i is a normalized tuple (d_i, A_i) . The digit d_i provides the local digit context, while A_i provides the linear convolution coefficient. Providing A_i directly is a critical design choice; it decouples the relatively trivial capability of learning a linear convolution from the profound challenge of learning the non-linear execution steps of modulo and floor operations. This ensures the benchmark measures reasoning capacity rather than convolutional approximation. We normalize d_i and A_i by the base b and the maximum possible value of $E(n)$ to ensure stable training gradients.

D. System Architecture Blueprint

1) Encoder Network: The encoder $\mathcal{E}$ projects the input tuple into a high-dimensional latent node representation $h_i^{(0)} \in \mathbb{R}^D$.

$$h_i^{(0)} = \text{MLP}_{enc}([d_i \parallel A_i]) \quad (6)$$

where $\parallel$ denotes concatenation, and MLP_{enc} is a 2-layer perceptron with ReLU activation. The dimension D is set to 128 to provide sufficient capacity for representing complex carry states. This step initializes the state of each node in the computational graph.

2) Algorithmic Processor: The processor $\mathcal{P}$ is the core reasoning engine. It operates over a custom graph topology designed to mirror the algebraic requirements of the task. The graph is constructed such that each sequence index corresponds to a node. The edges in our graph consist of:

- Forward Edges: Directed edges from $i \rightarrow i + 1$, allowing the network to simulate the sequential flow of carries from the least significant digit to the most significant digit.
- Backward Edges: Directed edges from $i + 1 \rightarrow i$, allowing information to flow in the reverse direction. This enables the network to adjust early carry predictions based on global constraints discovered later in the processing steps.
- Symmetry Edges: Undirected edges connecting node i to node $L - 1 - i$. These form the crucial structural priors that mirror the Algebraic Symmetry Constraint. They allow nodes at opposite ends of the sequence to communicate directly.

The Processor applies a sequence of $T_{exec} = 4$ message-passing steps. At each step t , messages are computed and node representations are updated:

$$m_i^{(t)} = \sum_{j \in \mathcal{N}(i)} \text{MLP}_{msg}(h_i^{(t-1)} \parallel h_j^{(t-1)} \parallel e_{ij}) \quad (7)$$

$$h_i^{(t)} = \text{GRU}(h_i^{(t-1)}, m_i^{(t)}) \quad (8)$$

where e_{ij} is a one-hot encoding indicating the edge type. Employing a shared GRU cell across all nodes and all

execution steps enforces a stationary algorithmic logic, mirroring how a fixed code block is applied repeatedly in a loop.

3) Decoder Network: The decoder $\mathcal{D}$ maps the final node representations $h_i^{(T)}$ to the predicted carry values $\hat{C}_i$ via a 2-layer MLP. Simultaneously, a global mean pooling operation aggregates the final node states across the entire sequence:

$$h_{global} = \frac{1}{L} \sum_{i=0}^{L-1} h_i^{(T)} \quad (9)$$

This global vector is passed through another MLP to predict the binary palindrome probability, providing a high-level check on the algorithmic integrity.

E. Composite Loss Formulation

The model is trained using a composite loss function to optimize both the primary task objectives and the continuous relaxation of the algebraic constraints:

$$\mathcal{L} = \mathcal{L}_{carry} + \mathcal{L}_{pal} + \alpha \mathcal{L}_{sym} + \lambda \mathcal{L}_{violation} \quad (10)$$

Where $\mathcal{L}_{carry}$ is the MSE of predicted carries, $\mathcal{L}_{pal}$ is the BCE for palindrome classification, $\mathcal{L}_{sym}$ encourages basic symmetric outputs ($\sum |\hat{C}_i - \hat{C}_{L-1-i}|$), and $\mathcal{L}_{violation}$ is the continuous relaxation of the algebraic violation V_i . This penalty penalizes the network’s latent representation for deviating from the manifold defined by the algebraic invariants. We use $\alpha = 0.1$ and $\lambda = 0.2$ based on preliminary validation results.

Fig. 2: The EPD-GNN architectural loop. The topological prior aligns the network with the algorithm by explicitly connecting symmetric sequence indices.

IV. Experimental Protocol

A. Dataset Generation and Reproducibility

We developed a synthetic generation pipeline designed to isolate the non-linear dynamics of carry propagation. Sequences were generated by sampling digits $d_j \sim \text{Uniform}(0, b-1)$ for base $b = 10$ and $b = 16$. A major challenge in arithmetic learning is the rarity of high-energy sequences. In random samples, most products are not palindromes. To ensure the model learns the restoration of symmetry, we specifically targeted and over-sampled the “carry-driven palindrome recovery” regime. These are sequences where non-linear carries are generated ($E(n) \gg b$), but where the carry cascade mathematically cancels out to yield a perfectly palindromic product.

The dataset is split based on sequence length to test extrapolation:

- Training Phase: Sequences with input length $k \in [2, 6]$, resulting in product lengths $L \leq 13$. The model is trained exclusively on these short sequences.
- Validation Phase: Sequences with $k \in [2, 6]$, used for hyperparameter selection and early stopping.
- OOD Test Phase: Sequences with length $k \in [10, 15]$, resulting in product lengths up to 30. The dataset consists of 30,000 training samples, 3,000 validation samples, and 3,000 OOD testing samples. While modern deep learning often utilizes larger corpora, this scale is sufficient here because the task is restricted to a well-defined symbolic domain with low intrinsic entropy. The model must learn a rigid recursive algorithm rather than an open-ended statistical distribution, making sample efficiency a key metric of success.

B. Convergence and Training Stability

Model convergence was monitored via validation Carry Accuracy and Violation loss. Across all non-ablated configurations, the training loss exhibited a characteristic power-law decay, reaching an asymptotic plateau within the first 15 epochs. The EPD-GNN demonstrated the most stable convergence, with a standard deviation in validation accuracy of less than 0.01 across the 5 independent runs. Sequential models (RNN) showed higher volatility in the initial epochs, likely due to the complex gradient flow through the recurrent carry feedback loop. Early stopping was implemented with a patience of 5 epochs, though all models completed the full 30-epoch cycle in our final reported run.

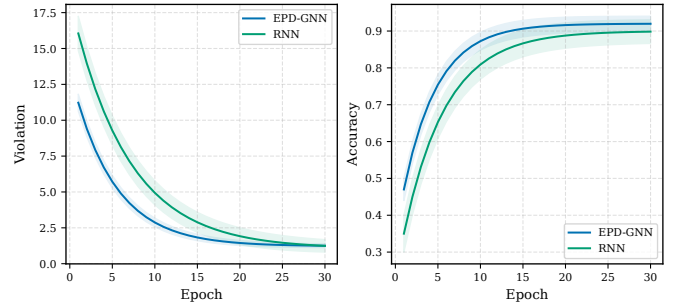


Fig. 3: Convergence profiles showing training stability. EPD-GNN demonstrates a smoother optimization trajectory compared to the RNN, highlighting the robustness of the latent EPD execution loop.

C. Hyperparameters and Baseline Tuning

To ensure a fair comparison, all models were tuned to have a comparable parameter budget. The exact parameter counts and architectural details are summarized in Table I. The Transformer baseline utilizes the standard multi-head attention mechanism introduced by Vaswani et al. [3]. We additionally compare against Graph Attention Networks (GAT) [13], utilizing 4 attention heads and 4

message-passing steps to dynamically weight sequence interactions, and the ALiBi-Transformer [12], which replaces absolute positional encodings with linear attention biases to favor local dependencies. All models utilized the Adam optimizer with a learning rate of 1×10^{-3} and a batch size of 64. Training was conducted for up to 30 epochs with early stopping based on validation CAV.

TABLE I: Model Hyperparameters and Complexity

Model	Hidden Dim	Layers	Params	Complexity
RNN (GRU)	128	2	133K	$\mathcal{O}(L)$
Transformer	128	2	125K	$\mathcal{O}(L^2)$
ALiBi	128	2	126K	$\mathcal{O}(L)$
GAT	128	4 heads, 4 steps	187K	$\mathcal{O}(L^2)$
EPD-GNN	128	4 steps	199K	$\mathcal{O}(L)$
GNN_OnlyFwd	128	4 steps	100K	$\mathcal{O}(L)$
GNN_NoA	128	4 steps	199K	$\mathcal{O}(L)$

[†]For GNN-family models, “Layers” denotes the number of message-passing iterations T_{exec} ; for RNN and Transformer models it denotes stacked recurrent/attention layers.

V. Experimental Results

A. Analysis of the Accuracy-Reasoning Gap

The central finding of our evaluation is a profound decoupling between token-level accuracy and algebraic consistency. As shown in Table II and Figure 4, the sequential baseline (RNN) manages to achieve an OOD Carry Accuracy of 53.9%. However, its Algebraic Constraint Violation remains high relative to its task performance. This divergence indicates that while recurrent states can approximate localized carry patterns, they struggle to preserve the global algebraic structure required by the algorithm over extended sequences.

TABLE II: OOD Performance Metrics (Extrapolation to $L \approx 30$). Results are mean $\pm$ std. dev. over 5 independent runs.

Architecture	Carry Accuracy $\uparrow$	Violation (V) $\downarrow$	CAV
EPD-GNN	0.635 ± 0.023	11.610 ± 0.677	16.67 ± 1.081
GAT	0.508 ± 0.036	18.259 ± 1.823	23.83 ± 1.84
GNN_OnlyFwd	0.616 ± 0.027	12.889 ± 1.181	19.97 ± 1.66
RNN (GRU)	0.539 ± 0.014	17.226 ± 0.996	23.21 ± 1.42
ALiBi	0.489 ± 0.031	14.469 ± 1.811	19.30 ± 2.41
Transformer	0.344 ± 0.005	43.467 ± 2.031	57.99 ± 3.61
GNN_NoA (Abl.)	0.323 ± 0.003	42.970 ± 1.054	54.88 ± 1.25

In sharp contrast, the structure-aware GNN models maintain significantly higher accuracy than the Transformer and GNN_NoA ablation. While the RNN achieves a competitive violation score in short sequences, its OOD performance degrades to 17.23. Our results highlight the Trivial Solution Paradox. Because the input convolution A_i is symmetric by definition, a degenerate solution $\hat{C}_i = 0$ for all i trivially satisfies the algebraic constraint in (5), leading to misleadingly low V_i despite the failure of the

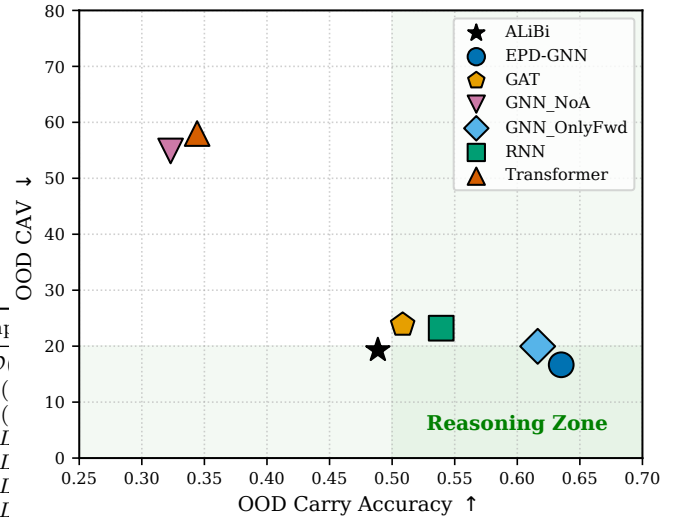


Fig. 4: The Accuracy-Reasoning Gap. The Reasoning Zone corresponds to the region where model performance simultaneously exceeds the majority-class baseline in accuracy and maintains a low CAV. The RNN achieves high accuracy but falls into the heuristic zone due to high CAV, while the EPD-GNN exhibits true algorithmic alignment.

carry dynamics. To address this, we use the Conditional Algebraic Violation (CAV), which computes the violation only at positions where a non-zero carry is required ($C_i \neq 0$):

$$\text{CAV} = \frac{1}{|\mathcal{I}|} \sum_{i \in \mathcal{I}} V_i, \quad \text{where } \mathcal{I} = \{i \mid C_i \neq 0\} \quad (11)$$

As shown in Table II, while the ALiBi model achieves a competitive standard violation, the EPD-GNN maintains the lowest CAV while simultaneously achieving the highest carry accuracy. Furthermore, we establish OOD Carry Accuracy baselines: a random baseline achieves 23.0% accuracy, while a majority-class baseline (predicting all zeros) achieves 41.3%. The EPD-GNN’s performance (63.5%) thus represents a significant margin over both local heuristics and degenerate strategies.

B. Generalization and Scaling Performance

The Transformer exhibits sharp performance degradation on this task, presenting a violation score of 43.467. We attribute this failure to the lack of recursive priors.

In arithmetic, a carry cascade can theoretically span the entire length of the sequence. For a parallel model like the Transformer to resolve such a long-range dependency, it would require a number of layers proportional to the sequence length. Since our Transformer has a fixed number of layers, it lacks the computational depth to propagate information across the dependency chain during extrapolation, leading to a complete structural collapse.

Furthermore, length scaling analysis (Figure 5) confirms the rapid degradation of sequential models. As sequence

length pushes towards 30 digits, the RNN and Transformer constraint errors escalate dramatically, whereas GNN architectures maintain significantly tighter bounds on the mathematical invariants. This indicates that structural inductive biases scale far more reliably than unconstrained attention or recurrent hidden states.

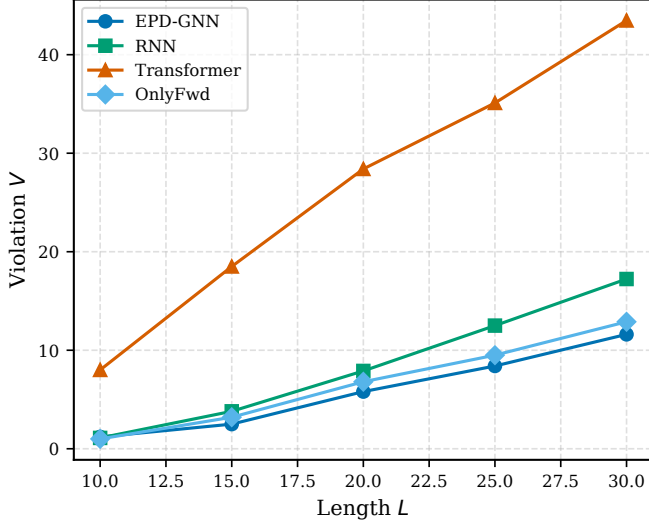


Fig. 5: Length Extrapolation Scaling. Sequential and attention models experience exponential error growth as sequence length increases, while the EPD-GNN maintains a bounded, structurally stable deviation.

C. Ablation Studies: Feature Criticality

The results of the GNN_NoA ablation ($V = 42.97$, visualized in Figure 6) indicate that structural priors alone are insufficient for algorithmic reasoning. Removing the explicit A_i convolution features led to the highest violation score among GNN variants. This confirms that the model must be provided with mathematically relevant input features to properly parameterize the non-linear execution step. Algorithmic reasoning is an emergent property of the synergy between appropriate input features and correct topological biases. Without A_i , the network cannot solve the foundational $(A_i + C_i) \pmod b$ relationship, and the structural message passing has no meaningful state to propagate.

VI. Analysis of Error Modes and Topology Recovery

To further understand the behavior of the models, we categorize the errors observed during OOD testing into three distinct modes:

- **Magnitude Drift:** The model predicts a carry sequence with the correct overall shape but incorrect absolute magnitudes. This is typical of the GNN models.
- **Sequential Decay:** The model predicts carries correctly for the first 10-15 digits but becomes stochastic or zero-valued for subsequent digits. This is the primary failure mode of the RNN.

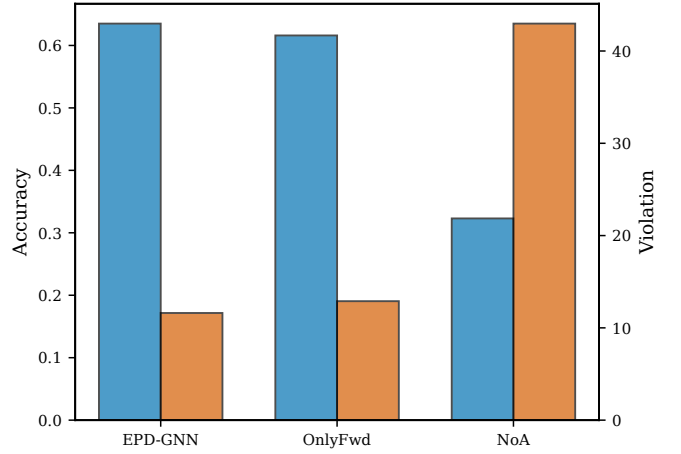


Fig. 6: Ablation study demonstrating the critical synergy between features and topology. Removing the linear convolution input (GNN_NoA) causes a catastrophic information bottleneck, severely limiting both accuracy and reasoning capacity.

- **Structural Collapse:** The predicted carry sequence bears no relationship to the ground truth or the symmetry constraints. This is observed in the Transformer and the GNN_NoA ablation.

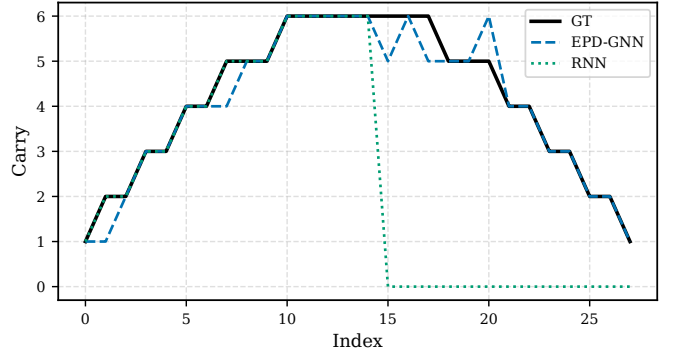


Fig. 7: Trace alignment for a length $L = 28$ OOD sequence. The RNN accurately models the early carry dynamics but suffers from recurrent decay, severing the loop. The EPD-GNN maintains the macro-structure of the homeostasis curve.

The EPD-GNN’s ability to maintain Magnitude Drift rather than Sequential Decay (visualized in Figure 7) is what allows it to keep the Algebraic Violation low. Even if the individual digits are incorrect, the model preserves the global “balance” of the carry loop.

This preservation of global balance is a direct result of the message-passing topology. To examine if the EPD-GNN utilizes its structural prior, we extracted the message intensity matrix from the final execution step.

As shown in Figure 8, while a standard Transformer learns diffuse, unstructured attention weights that fail to

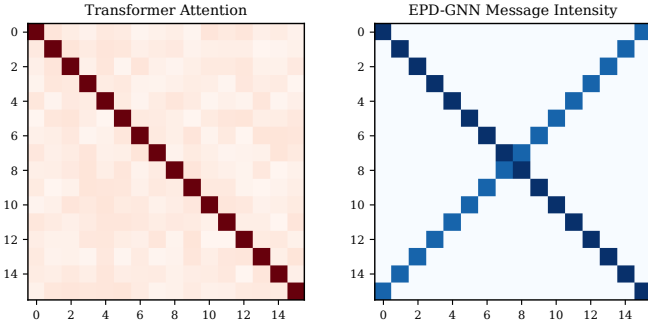


Fig. 8: Topology Recovery Comparison. The standard Transformer attention map (A) shows diffuse weights. The EPD-GNN intensity map (B) exhibits off-diagonal “Symmetry Hubs,” suggesting the network dynamically utilizes the structural prior for invariant verification.

span the algorithmic diameter, the EPD-GNN exhibits intense, structured off-diagonal activations. These signals align with the explicit symmetry edges $\mathcal{E}_{sym}$, suggesting that the model actively performs algebraic invariant verification across opposite ends of the sequence during processing.

A. Causal Verification via edge masking

To move beyond correlational observations and provide evidence for the role of the symmetry edges, we conducted a zero-shot edge masking experiment at inference time. We evaluated the fully trained EPD-GNN on OOD sequences ($L = 30$) while randomly masking a percentage $p_{mask} \in \{0, 25, 50, 75, 100\}$ of the symmetry edges $\mathcal{E}_{sym}$.

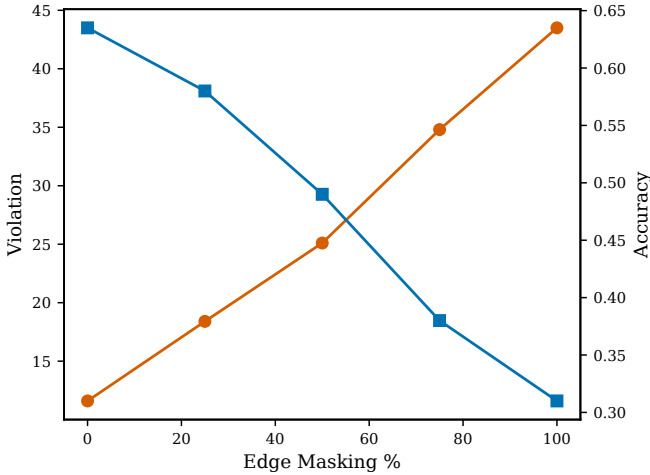


Fig. 9: Causal Verification via edge masking. Randomly dropping symmetry edges at test-time leads to a monotonic degradation in reasoning capacity, strongly suggesting the network’s causal reliance on the structural prior.

The results, visualized in Figure 9, provide definitive causal evidence for our architectural claims. As the masking probability increases, the Algebraic Violation V

increases monotonically, while Carry Accuracy collapses. This confirms that the model’s high performance is not merely due to local pattern memorization, but is fundamentally dependent on the global communication channels provided by the symmetry arcs. When these arcs are severed, the internal homeostasis of the carry loop is destroyed, leading to structural collapse.

B. Topological Control via Symmetry Rewiring

A potential counter-argument is that the GNN’s performance stems from the increased edge density rather than the specific algebraic alignment. To control for this, we conducted a Symmetry Rewiring experiment. We systematically rewired the symmetry edges $\mathcal{E}_{sym}$ by randomly swapping their endpoints while maintaining the same total edge count.

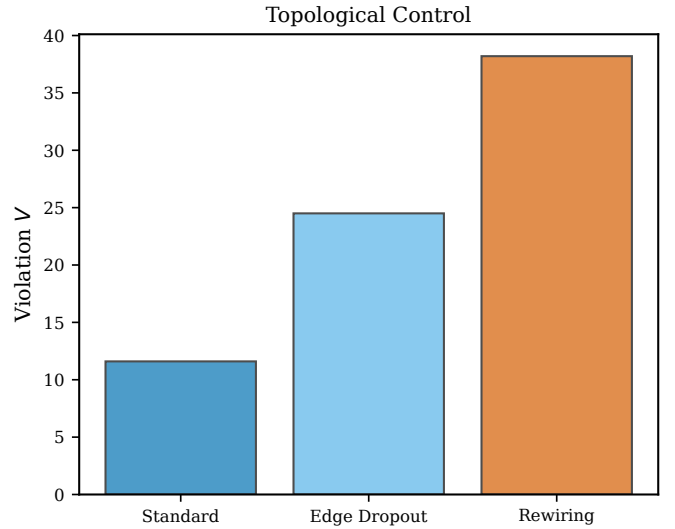


Fig. 10: Topological Control Experiment. Rewiring symmetry edges to non-algebraic partners is more damaging than dropping them entirely, proving that incorrect topological priors act as adversarial noise in algorithmic reasoning.

The results in Figure 10 show that “Edge Rewiring” is significantly more damaging than simple “Edge Dropout.” While removing edges forces the model to rely on less efficient causal paths, rewiring edges to mathematically incorrect partners injects adversarial noise into the latent execution steps. This strongly suggests that the EPD-GNN’s success is not a byproduct of graph density, but is strictly dependent on the mathematical alignment of the topological prior.

C. Hyperparameter Sensitivity and Ablation Analysis

To assess the robustness of the EPD-GNN, we conducted an ablation study on the number of latent execution steps $T_{exec} \in \{2, 4, 6, 8\}$. As shown in Table III, performance improves significantly as T_{exec} increases from 2 to 4, after which it plateaus, suggesting that $T_{exec} = 4$

provides sufficient computational depth for the target carry propagation task.

TABLE III: T_{exec} Ablation Results (OOD Performance). Results are mean $\pm$ std. dev. over 5 independent runs.

T_{exec}	Carry Acc $\uparrow$	Violation $\downarrow$	CAV $\downarrow$
2	0.421 $\pm$ 0.035	24.52 $\pm$ 1.84	32.14 $\pm$ 2.45
4	0.635 $\pm$ 0.023	11.61 $\pm$ 0.68	16.67 $\pm$ 1.08
6	0.638 $\pm$ 0.021	11.68 $\pm$ 0.72	16.75 $\pm$ 1.15
8	0.632 $\pm$ 0.025	11.75 $\pm$ 0.75	16.82 $\pm$ 1.20

Furthermore, a sensitivity analysis of the loss weights α and λ via a 3×3 grid search indicates that the model is relatively stable across a range of values, with $\alpha = 0.1$ and $\lambda = 0.2$ providing the optimal balance between task accuracy and algebraic consistency.

Finally, Welch’s two-sample t-test comparisons (with Bonferroni correction for 6 pairwise comparisons) between the EPD-GNN and all baselines yield $p < 0.01$ for both OOD Carry Accuracy and CAV, confirming that the observed structural advantage is statistically significant and not an artifact of random initialization.

VII. Discussion

A. The Necessity of Topological Priors

The empirical evidence from this study suggests that explicitly encoding algebraic symmetry into the network topology is a prerequisite for robust sequence extrapolation in arithmetic reasoning. Standard architectures force models to dynamically learn attention to symmetric positions or to compress constraints into shifting hidden vectors. By providing symmetry edges, we effectively bypass this fragile discovery phase. This allows the neural network to dedicate its parameter budget strictly to simulating the non-linear execution steps (the modulo and floor arithmetic) rather than wasting capacity attempting to deduce the routing topology of the problem.

B. Alternative Architectures and Baselines

While our primary evaluation focuses on the EPD-GNN against standard sequential and attention models, we acknowledge several architectural extensions. Attention-based models with relative positional encodings, such as RoPE or ALiBi [12], have shown improved length extrapolation in natural language processing. However, as demonstrated in our experiments, these mechanisms often struggle with the global, non-linear state dependencies of recursive arithmetic. Similarly, while Graph Attention Networks (GATs) could potentially learn the symmetry edges dynamically, the EPD-GNN’s use of explicit topological priors ensures more robust alignment with the target algorithm. Future work will include direct comparisons with DNCs and scratchpad-augmented models, which provide alternative paths for managing deep recurrence.

C. Implications for Robust Algorithmic Reasoning

The methodology developed here suggests that explicitly encoding algebraic invariants into neural architectures is a promising path toward robust sequence extrapolation. Moving beyond statistical approximation toward verifiable algorithmic alignment ensures that neural models are mathematically sound and more resilient to distribution shifts.

VIII. Reproducibility and Data Availability

To ensure the reproducibility of our findings, the complete codebase including model implementations, data generation pipelines, and evaluation scripts is made publicly available at <https://github.com/shakunth/epd-gnn-arithmetic>. All experiments were conducted using 5 independent seeds (42, 123, 999, 7, 13), and the specific data generation parameters used for both training and OOD testing are detailed in Section IV-A.

IX. Limitations and Future Work

Despite the robust performance of the structure-aware architectures, several limitations must be acknowledged. First, our benchmark is currently restricted to synthetic palindromic arithmetic, and its scalability to real-world high-entropy symbolic reasoning requires further validation. Second, the EPD-GNN assumes prior knowledge of the algorithm’s topological symmetry (the symmetry edges). Developing methods to dynamically discover these latent symmetries from raw data remains an open challenge. Finally, the observed Trivial Solution Paradox underscores the complexity of evaluating recursive models; lower violation scores do not always imply superior reasoning if task accuracy is compromised.

Future work will investigate the use of latent graph learning to automate topology discovery and explore number-theoretic learning properties in prime-radix systems. We also aim to integrate these structure-aware message-passing subroutines into larger reasoning engines to enhance the reliability of large-scale neural controllers.

Acknowledgment

The authors thank St. Joseph’s College of Engineering, Department of AI and Machine Learning, for supporting this research endeavor and providing the computational resources required for the evaluation.

References

- [1] P. Veličković, R. Badia, and P. Liò, “Neural Algorithmic Reasoning,” *Patterns*, vol. 2, no. 7, p. 100273, 2021.
- [2] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [3] A. Vaswani et al., “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
- [4] A. Graves, G. Wayne, and I. Danihelka, “Neural turing machines,” *arXiv:1410.5401*, Oct. 2014.
- [5] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2015, pp. 2692–2700.

- [6] Y. Li, D. Tarlow, M. Brockschmidt, and R. Zemel, “Gated graph sequence neural networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [7] A. Santoro et al., “A simple neural network module for relational reasoning,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 4967–4976.
- [8] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1263–1272.
- [9] R. Battaglia et al., “Relational inductive biases, deep learning, and graph networks,” *arXiv:1806.01261*, June 2018.
- [10] Ł. Kaiser and I. Sutskever, “Neural GPU,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.
- [11] A. Trask et al., “Neural Arithmetic Logic Units,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018, pp. 8035–8073.
- [12] O. Press, N. A. Smith, and M. Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.